

JDBC

Java Database Connectivity

Kenapa Butuh JDBC?

- Aplikasi Java menyimpan data di objek (dalam RAM), sehingga data akan hilang saat program ditutup
- Database digunakan untuk menyimpan data secara permanen, tetapi database menggunakan bahasa SQL untuk mengelola datanya
- JDBC berperan sebagai penghubung antara aplikasi Java dan database

Fungsi JDBC:

- Menjalankan perintah SQL dari aplikasi Java
- Menghubungkan Java dengan berbagai RDBMS seperti MySQL, PostgreSQL, dan Oracle Database
- Sudah tersedia di Java Standard Library, sehingga tidak perlu framework tambahan



RDBMS vs OODBMS

RDBMS (Relational)

- Data disimpan dalam bentuk tabel (baris dan kolom)
- Tabel bersifat normalized (ternormalisasi) sehingga data tidak banyak diulang
- Setiap tabel memiliki primary key sebagai identitas unik data

Contoh: MySQL, PostgreSQL, Oracle

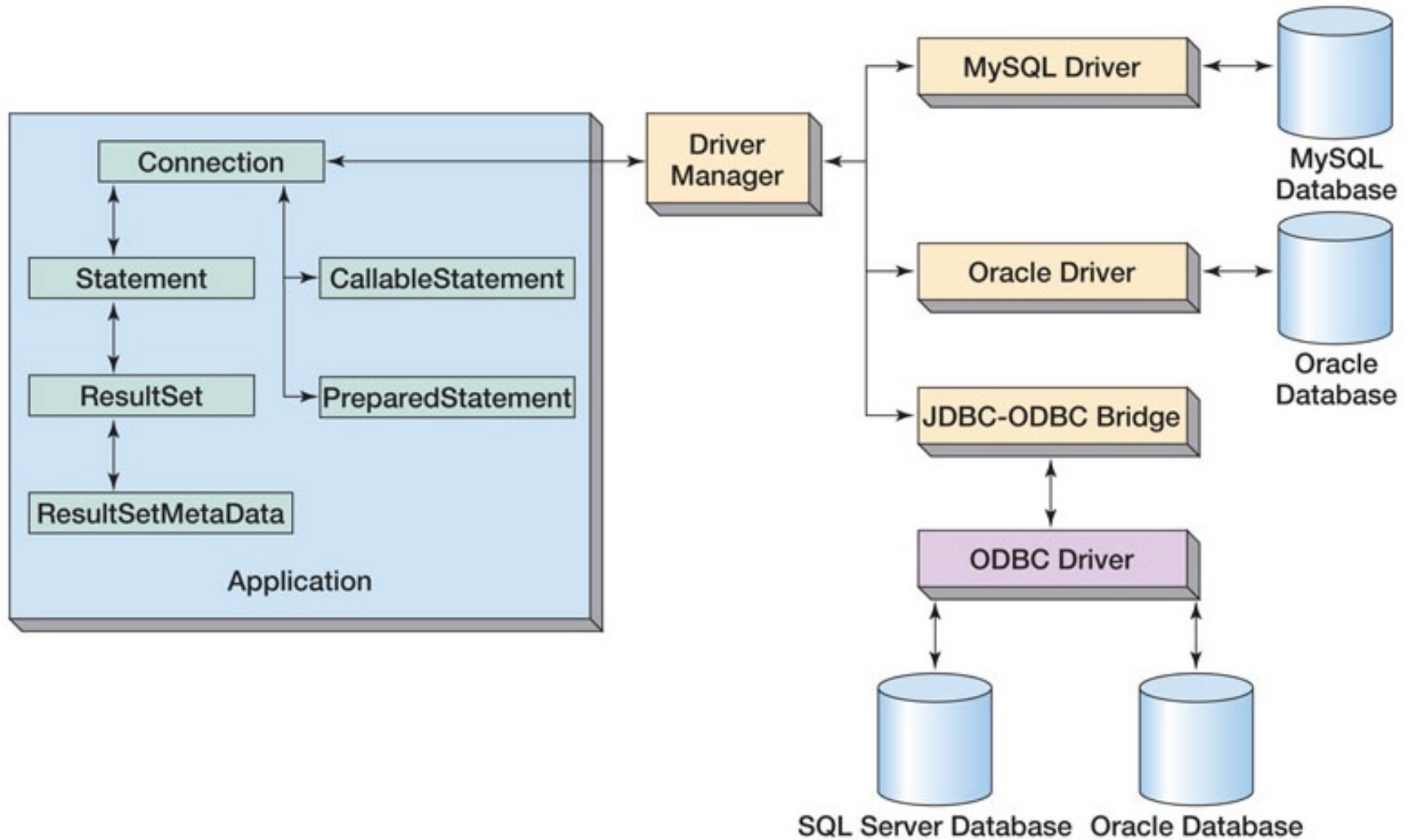
OODBMS (Object-Oriented)

- Data disimpan dalam bentuk object, mirip seperti class pada Java
- Object dapat saling berelasi melalui reference
- Struktur data tidak harus ternormalisasi seperti pada RDBMS
- Setiap object memiliki object identifier sebagai identitas unik

Contoh: db4o, ObjectDB

Dalam mata kuliah ini kita menggunakan RDBMS (MySQL). JDBC digunakan sebagai API Java untuk mengakses database.

Arsitektur JDBC



Alur Database Programming dengan JDBC

1 Load Driver

```
Class.forName("com.mysql.jdbc.Driver");
```

2 Buat Koneksi

```
Connection conn = DriverManager.getConnection(url, user, pass);
```

3 Buat Statement

```
Statement stmt = conn.createStatement();
```

4 Eksekusi SQL

```
ResultSet rs = stmt.executeQuery(sql); // atau executeUpdate()
```

5 Proses Hasil Query

```
while (rs.next()) { ... }
```

6 Tutup Resource

```
stmt.close(); conn.close();
```

** Langkah 1 bisa dihilangkan karena pada JDBC 4.0+ driver biasanya sudah auto-load jika file .jar driver ada di classpath.*

Contoh Kode JDBC - Koneksi & Query SELECT

```
String url  = "jdbc:mysql://localhost:3306/exercise";
String user = "root";
String pass = "";

Connection conn = DriverManager.getConnection(url, user, pass);
Statement stmt = conn.createStatement();

String sql = "SELECT * FROM product";
ResultSet rs = stmt.executeQuery(sql);

while (rs.next()) {
    int    id    = rs.getInt("id");
    String name  = rs.getString("name");
    double price = rs.getDouble("price");
    System.out.println(id + " | " + name + " | " + price);
}

rs.close();
stmt.close();
conn.close();
```

Setup: Database & Konfigurasi Koneksi

Studi kasus: pada contoh ini, kita menggunakan tabel product di database MySQL melalui XAMPP

Schema database (buat via phpMyAdmin):

Field	Tipe	Keterangan
id	INT	PK, auto_increment
name	VARCHAR(50)	—
price	DOUBLE	—

Class koneksi (JDBC.java):

```
private static final String URL = "jdbc:mysql://localhost:3306/exercise";
private static final String USER = "root";
private static final String PASS = "";

public Connection connect() throws Exception {
    return DriverManager.getConnection(URL, USER, PASS);
}
```

Model Class & Mapping ResultSet → Object

- Hasil query SELECT di JDBC berupa object ResultSet
- Data pada ResultSet perlu di-mapping secara manual ke atribut pada class Java
- Fungsi toModel(ResultSet rs) digunakan untuk mengubah satu baris data menjadi satu object
- Fungsi get() biasanya digunakan untuk mengambil seluruh data dan menyimpannya ke dalam ArrayList<Product>

Struktur Product.java:

```
public class Product extends JDBC {  
    private int id;  
    private String name;  
    private double price;  
  
    public Product() {  
  
    }  
}
```

Fungsi toModel():

```
Product toModel(ResultSet rs) {  
    try {  
        return new Product(  
            rs.getInt("id"),  
            rs.getString("name"),  
            rs.getDouble("price")  
        );  
    } catch (Exception e) {  
        setMessage(e.getMessage());  
    }  
    return null;  
}
```


Fungsi get() - Mengambil Semua Data sebagai ArrayList

- Fungsi get() digunakan untuk menjalankan query SELECT * pada tabel database
- Setiap baris hasil query pada ResultSet akan dikonversi menjadi object menggunakan fungsi toModel()
- Hasil akhirnya berupa ArrayList<Product> yang siap digunakan di controller, JSP, atau bagian aplikasi lainnya

```
public ArrayList<Product> get() {  
    ArrayList<Product> res = new ArrayList<>();  
    try {  
        ResultSet rs = getData("SELECT * FROM product");  
        while (rs.next()) {  
            res.add(toModel(rs));  
        }  
    } catch (Exception e) {  
        setMessage(e.getMessage());  
    }  
    return res;  
}
```

- Pola ini bisa digunakan untuk semua model, cukup menyesuaikan nama tabel dan atributnya

Tampilkan Data - Alur MVC

Alur: doGet() di Controller → panggil get() dari Model → kirim ke JSP via setAttribute() → JSP menampilkan data ke halaman web

ProductController.java - doGet():

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String menu = request.getParameter("menu");
    if (request.getParameterMap().isEmpty()) { //view menu Model
        request.setAttribute("title", "Daftar Produk");
        ArrayList<Product> prods = new Product().get();
        request.setAttribute("list", prods);
        request.getRequestDispatcher("view_product.jsp").forward(request, response);
        request.getSession().removeAttribute("msg");
    }
```

view_product.jsp (loop ArrayList):

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.ArrayList, models.Product" %>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>${title}</title>
</head>
<body>
<h3>${title}</h3>
<table border="1">
<tr>
<th>No</th>
<th>Nama</th>
<th>Harga</th>
</tr>
<%
    ArrayList<Product> prods = (ArrayList<Product>)request.getAttribute("list");
    int i = 1;
    for (Product p : prods) {
%>
<tr>
<th><%= i++ %></th>
<th><%= p.getName() %></th>
<th><%= p.getPrice() %></th>
</tr>
<% } %>
</table>
</body>
</html>
```

Coba akses:

localhost:8080/[project]/[servlet]

Query Custom - Plain Object (bukan Model)

- Tidak semua hasil query cocok di-mapping ke class Model. Jika query menghasilkan kolom tambahan seperti agregasi (COUNT, AVG, dll.), hasilnya tidak perlu dipaksakan menjadi object Model
- Pada kasus seperti ini, data bisa disimpan sebagai `ArrayList<Object>`. Nilai data diakses berdasarkan posisi kolom hasil query

```
// Contoh query agregasi
String sql = "SELECT COUNT(*), AVG(price) FROM product";

// Akses hasilnya:
ArrayList<Object> row = result.get(0);
row.get(0);    // → COUNT(*)
row.get(1);    // → AVG(price)
```

Fungsi yang dibutuhkan di `Product.java`:

- `toRow(ResultSet rs)` → untuk mengubah satu baris `ResultSet` menjadi `ArrayList<Object>`
- `query(String sql)` → untuk menjalankan query SQL custom dan mengembalikan hasil dalam bentuk `ArrayList<ArrayList<Object>>`

Insert Data

Dua tahap: (1) GET → menampilkan form kosong, (2) POST → memproses dan menyimpan data yang dikirim dari form

doGet() – menampilkan form:

```
if ("add".equals(menu)) {
    request.setAttribute("title",
        "Tambah Produk");
    dispatcher("form_product.jsp")
        .forward(req, res);
}
```

form_product.jsp:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="models.Product" %>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>${title}</title>
</head>
<body>
<h4>Form ${title}</h4>
<%
    Product prod = null;
    if (request.getAttribute("product") != null) prod = (Product)request.getAttribute("product");
%>
<form class="border" style="padding:20px" method="POST" action="product${action}">
    <div>
        <label>Nama</label>
        <input type="text" name="name" class="form-control" value="<%= prod == null ? "" : prod.getName() %>" required />
    </div>
    <div>
        <label>Harga</label>
        <input type="number" name="price" class="form-control" value="<%= prod == null ? "" : prod.getPrice() %>" required />
    </div>
    <div>
        <button class="btn btn-success">Simpan</button>
    </div>
</form>
</body>
</html>
```

doPost() - menyimpan data

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String action = request.getParameter("action");
    String id = request.getParameter("id");
    if (id == null) { //insert data
        Product p = new Product();
        p.setName(request.getParameter("name"));
        p.setPrice(Double.parseDouble(request.getParameter("price")));
        p.insert();
    }

    response.sendRedirect("product");
}
```

Update Data

Tiga tahap: (1) load data by id, (2) menampilkan form pre-filled (GET), (3) menyimpan perubahan data (POST)

doGet() - load data & buka form edit:

```
} else if ("edit".equals(menu)) { //edit menu
    request.setAttribute("title", "Edit Produk");
    request.setAttribute("action", "?id=" + request.getParameter("id"));
    Product p = new Product().find(request.getParameter("id"));
    if (p == null) {
        response.sendRedirect("product");
        return;
    }
    request.setAttribute("product", p);
    request.getRequestDispatcher("form_product.jsp").forward(request, response);
}
```

doPost() – menyimpan perubahan:

```
} else if (action == null) { //update data
    Product p = new Product();
    p.setId(Integer.parseInt(id));
    p.setName(request.getParameter("name"));
    p.setPrice(Double.parseDouble(request.getParameter("price")));
    p.update();
    request.getSession().setAttribute("msg", p.getMessage());
}
```

Delete Data & Method di Model

doPost() - menghapus data:

```
} else if ("del".equals(action)) { //delete data
    Product p = (Product)new Product().find(id);
    if (p != null) {
        p.delete();
    }
}
```

- Satu class Model biasanya berisi seluruh method CRUD seperti toModel(), get(), find(), insert(), update(), dan delete()
- Controller cukup memanggil method dari Model tanpa perlu menulis query SQL secara langsung

Semua method CRUD di Product.java:

```
public Product find(String id) {
    ResultSet rs = getData("SELECT * FROM product WHERE id = " + id);
    try {
        if (rs.next()) {
            return toModel(rs);
        }
    } catch (Exception e) {
        setMessage(e.getMessage());
    }
    return null;
}

public void insert() {
    runQuery("INSERT INTO product(name, price)"
            + " VALUES ('" + name + "', '" + price + "')");
}

public void update() {
    runQuery("UPDATE product SET name = '" + name + "', price = '" + price + "'"
            + " WHERE id = " + id);
}

public void delete() {
    runQuery("DELETE FROM product WHERE id = " + id);
}
```

Session

- Session adalah mekanisme yang digunakan web server untuk menyimpan data sementara antar halaman dan antar request
- Biasanya digunakan untuk menyimpan data seperti status login, data akun, atau isi keranjang belanja

Karakteristik session:

- | | | |
|---|-----------------------------|--|
| 1 | Dikelola oleh server | data session disimpan di sisi server, bukan di browser user |
| 2 | Memiliki timeout | session memiliki timeout yang diatur oleh server (default ±30 menit) |
| 3 | Session ID unik | server menggunakan Session ID untuk membedakan session tiap browser/user |
| 4 | Dihapus otomatis | session bisa terhapus otomatis saat browser ditutup |

API Session:

```
request.getSession().setAttribute("username", username); // simpan
request.getSession().getAttribute("username");           // ambil
request.getSession().removeAttribute("username");         // hapus satu
request.getSession().invalidate();                         // hapus semua (logout)
```

Contoh: Login dengan Session

Skenario:

- User harus login dulu sebelum bisa akses halaman /home
- Setelah login berhasil, username disimpan ke dalam session
- HomeServlet akan mengecek apakah session masih ada. Jika session tidak ditemukan, user akan diarahkan kembali ke halaman login

AuthServlet:

```
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String username = request.getParameter(name: "user");
    String password = request.getParameter(name: "pass");
    if ("admin".equals(anObject: username) && "admin123".equals(anObject: password)) {
        request.getSession().setAttribute(name: "username", value: username);
        response.sendRedirect(location: "home");
        return;
    }
    request.setAttribute(name: "msg", o: "username / password salah");
    request.getRequestDispatcher(path: "form_login.jsp").forward(request, response);
}
```

HomeServlet:

```
@WebServlet(name = "Home", urlPatterns = {"/home"})
public class HomeController extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        if (request.getSession().getAttribute(name: "username") == null) {
            response.sendRedirect(location: "auth");
            return;
        }

        String keyword = request.getParameter(name: "key");
        request.setAttribute(name: "keyword", o: keyword);
        request.getRequestDispatcher(path: "form_search.jsp").forward(request, response);
    }
}
```